



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

B.Tech Computer Science and Engineering
BCSE497J-PROJECT-1/MACSE698- Dissertation I
Project ID- 19727UG01

SecOps Agent for CAF/WAF compliant Cloud Deployments in CI/CD

Team members:

Akshat Gupta	(23BCE0958)
Gaurish Todi	(23BCI0262)
Bhumika Singh	(23BCE0196)

Faculty guide:

Dr. SM Farooq Sir (19727)

Verified Project Title & Abstract

PROJECT TITLE

Agent SecOps: An AI-Driven SecOps Agent for CAF/WAF-Aligned Infrastructure-as-Code Pipelines



SDG 9

Industry, Innovation
& Infrastructure



TRL 3

Experimental proof
of concept

EXPECTED OUTCOME (SNAPSHOT)

A working proof-of-concept AI agent that intercepts Terraform plans inside a GitHub Actions pipeline, scores them against Azure CAF governance and WAF pillars, and returns a pass / block verdict with a human-readable remediation report — before any resource reaches Azure, AWS, or GCP.

ABSTRACT

Modern cloud teams ship infrastructure using Terraform through CI/CD pipelines such as GitHub Actions, deploying directly into Azure, AWS, or GCP. Providers publish governance guidance — Microsoft's Cloud Adoption Framework (CAF) and Well-Architected Framework (WAF) — covering security, cost, reliability, and operational best practice, but this guidance is written as documentation, not enforceable code, so it is rarely checked automatically before deployment.

Existing policy-as-code tools (Checkov, OPA, Sentinel) only catch violations of rules someone has already written; they cannot reason about the broader architectural intent described in framework documentation. Agent SecOps proposes an AI agent placed between the Terraform plan stage and the deployment stage of the pipeline. It reads the generated plan, evaluates it against CAF/WAF principles using a policy-aware reasoning layer, and returns a pass, block, or remediate verdict with a plain-language justification — stopping non-compliant infrastructure before it is ever applied to the cloud.

Aim

- To design an AI SecOps agent that sits between Terraform (IaC) plan generation and CI/CD deployment (GitHub Actions) to Azure/AWS/GCP, automatically checking every proposed cloud resource against Microsoft's Cloud Adoption Framework (CAF) and Well-Architected Framework (WAF) before the pipeline is allowed to proceed to deployment.

Abstract

- Modern cloud teams ship infrastructure using Terraform through CI/CD pipelines such as GitHub Actions, deploying directly into Azure, AWS, or GCP. Providers publish governance guidance such as Microsoft's Cloud Adoption Framework (CAF) and Well-Architected Framework (WAF), but this guidance is written as documentation, not enforceable code, so it is rarely checked automatically before deployment. Existing policy-as-code tools (Checkov, OPA, Sentinel) only catch violations of rules someone has already written; they cannot reason about the broader architectural intent described in framework documentation.
- This project proposes SecOps Agent, an AI agent placed between the Terraform plan stage and the deployment stage of the pipeline. It reads the generated plan, evaluates it against CAF/WAF principles using a policy-aware reasoning layer, and returns a pass, block, or remediate verdict with a plain-language justification, stopping non-compliant infrastructure before it is ever applied to the cloud.

PROBLEM STATEMENT

Cloud Misconfiguration Is Still the #1 Preventable Risk

Provider frameworks (CAF/WAF) exist, but nothing stops an engineer from pushing a Terraform change that quietly violates them — until it's already live.

23%

of cloud security incidents trace directly to misconfigured resources

SentinelOne / IBM, 2025

~99%

of cloud security failures are projected to be the customer's fault — mostly config errors

Gartner

180+ days

average time a misconfiguration sits undetected before exploitation

Industry benchmarks, 2025–26

WHY IT PERSISTS

- CAF & WAF are narrative documentation — good judgement to read, nothing for a pipeline to execute.
- Static scanners (Checkov, tfsec, OPA, Sentinel) only catch violations of rules someone already wrote — they can't reason about undocumented edge cases or framework intent.
- Manual architecture review doesn't scale across dozens of daily pull requests, or across three different clouds at once.

Where Today's Policy-as-Code Tools Fall Short

Tool	Scope	How it decides	Limitation
Checkov / tfsec	Static IaC scanning	Pre-written security rules (Python/YAML)	Fixed rule library; no framework reasoning
HashiCorp Sentinel	Terraform Cloud/Enterprise only	Hand-authored Sentinel policies	Vendor-locked; policies written per-org
Open Policy Agent (OPA)	General purpose (K8s, IaC, APIs)	Rego rules against JSON input	Steep learning curve; rules still hand-written
Agent SecOps (proposed)	Terraform → any CI/CD → any cloud	LLM reasoning against CAF/WAF pillars + policy rules	Proof-of-concept stage (this project)

Core gap: static scanners enforce rules that already exist in code. Agent AI adds a reasoning layer that also checks against framework guidance that was never turned into a rule.

Agent SecOps, in One Sentence

An AI agent placed between “terraform plan” and “terraform apply” in the CI/CD pipeline, that gates every infrastructure change against Azure CAF and WAF principles before it can reach the cloud.



Reads the Plan

Parses the Terraform plan JSON to see exactly what resources are about to change — before they exist.



Reasons, Not Just Matches

Combines static policy rules with an LLM reasoning layer trained on CAF governance & WAF pillar guidance.



Gates the Pipeline

Blocks, warns, or approves the GitHub Actions run — no non-compliant resource reaches Azure/AWS/GCP.



Explains Itself

Returns a plain-language compliance report mapped to the exact CAF/WAF pillar that was violated.

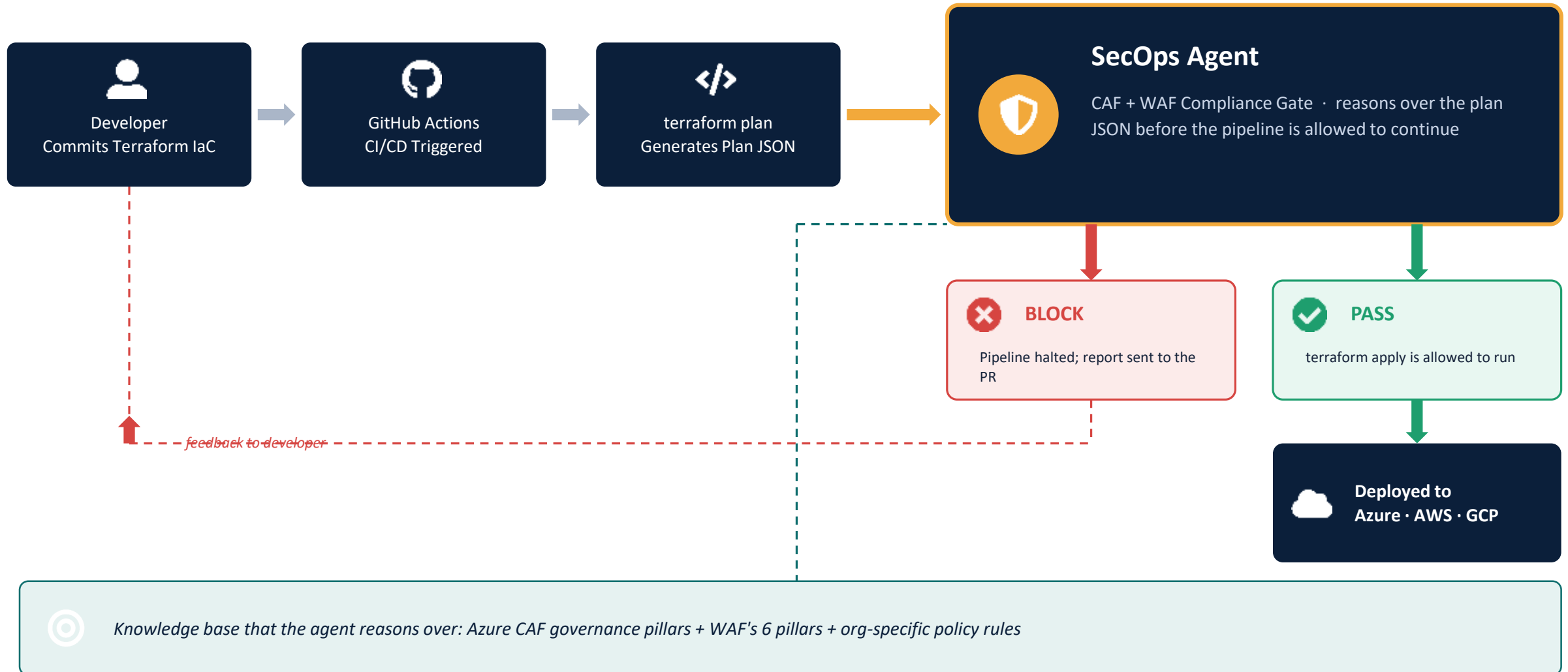
Objectives

- To build a proof-of-concept agent that parses Terraform plan JSON and evaluates each proposed resource against CAF governance rules and WAF's pillars (security, reliability, cost, operations).
- To integrate the agent as a gate inside a GitHub Actions workflow that blocks or passes deployment before terraform apply runs, and returns a human-readable compliance report for any violation found.
- To validate the approach across multiple cloud providers (Azure, AWS, GCP) so the gate logic remains cloud-agnostic.

Literature Review

S.No	Author(s) & Year	Title / Focus	Key Contribution	Research Gap Identified
1	Malul, Meidan, Mimran, Elovici, Shabtai (2024)	GenKubeSec: LLM-based Kubernetes misconfig detection	Fine-tuned LLM detects, localizes, reasons about & remediates K8s misconfigs; beats rule-based tools (Checkov, KubeLinter, Terrascan) on recall at equal precision	Limited to Kubernetes manifests; no CI/CD pipeline gating; not applied to Terraform or CAF/WAF
2	Vo, Dao & Fukuda (2025) IEEE COMPSAC	LLM-based Terraform code smell detection	Uses LLMs to detect Terraform code smells by generalizing beyond predefined rule-based heuristics	Detects smells but does not gate a live CI/CD pipeline or reason over CAF/WAF governance rules

Exactly Where the Agent Sits in the Pipeline



Research Gap

- Static policy-as-code tools (Checkov, tfsec, OPA, Sentinel) only detect violations of rules that have already been hand-written; they cannot reason about CAF/WAF guidance that was never codified into a fixed rule, and each requires separate, manually-maintained rule sets per cloud provider.
- Recent LLM-based work (GenKubeSec, Vo et al. 2025) shows LLMs can out-detect rule-based scanners for Kubernetes and Terraform code respectively, but no existing study embeds an LLM reasoning agent directly as a CI/CD pipeline gate that checks Terraform plans against CAF/WAF governance before deployment across multiple clouds — this is the specific gap Agent AI addresses.

SDG 9 – Industry, Innovation & Infrastructure



Aligned with SDG 9



SDG 9

**Industry, Innovation
and Infrastructure**

*Official UN Sustainable
Development Goal*

RELEVANT TARGET

Target 9.5 — “Enhance scientific research and upgrade the technological capabilities of industrial sectors ... encouraging innovation and substantially increasing the number of R&D workers, and public and private R&D spending.”

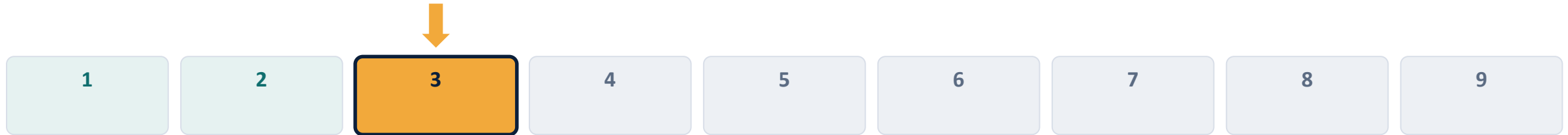
HOW Agent AI CONTRIBUTES

- Builds resilient digital infrastructure by preventing insecure or non-compliant cloud resources from ever being provisioned.
- Applies AI-based innovation to an industrial process (DevOps governance) that today relies on manual review.
- Supports sustainable infrastructure practice by enforcing cost- and resource-efficiency pillars (WAF) automatically at scale.

TRL – 3 (Experimental Proof of Concept)



Current Stage: TRL 3



TRL scale (NASA / ISO 16290 methodology, adapted for software)

TRL 3 — Analytical & Experimental Proof of Concept

- Critical function (an AI reasoning agent judging IaC plans against a governance framework) has been analytically defined and validated through research, not yet built end-to-end.
- Feasibility established by studying existing policy-as-code engines (OPA, Sentinel, Checkov) and confirming their static-rule limitation — the specific gap Agent AI is designed to close.
- Terraform's JSON plan output, GitHub Actions' workflow hooks, and Azure CAF/WAF documentation have all been reviewed as concrete integration points.
- No working prototype yet — that is the target of the next phase (TRL 4: component validation in a lab environment).

WHY THIS MEETS THE REVIEW-1 BAR

Review-1 requires TRL 3 or above. Agent AI clears this because the core concept has moved past pure ideation — it rests on a documented, testable proof-of-concept: a defined input (Terraform plan JSON), a defined decision logic (CAF/WAF pillar checks), and a defined output (pass/block verdict).

References

- [1] E. Malul, Y. Meidan, D. Mimran, Y. Elovici, and A. Shabtai, "GenKubeSec: LLM-Based Kubernetes Misconfiguration Detection, Localization, Reasoning, and Remediation," *arXiv preprint arXiv:2405.19954*, 2024.
- [2] Q.-H. Vo, H. Dao, and K. Fukuda, "Harnessing the Power of LLMs for Code Smell Detection in Terraform Infrastructure as Code," in *Proc. IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC)*, 2025, pp. 533-542.